

- Основные понятия сетей
 - GSM - сотовая сеть
 - Другие виды сетей
 - Виды топологий сети
 - Основные понятия
 - Примеры сетевых протоколов:
 - Сетевой уровень
 - IP-адресация
 - Классификация IP-адресов
 - Маршрутизация на сетевом уровне
- Физические пути соединения
- стек протоколов
- Физический уровень: более подробно
 - Передача сигналов
 - Роль драйверов
 - Идентификация устройств
- Сетевой уровень и IP-протокол
 - Сетевой протокол
 - Маршрутизация данных
 - IP-адресация
 - IPv4 и IPv6
 - Формат записи IP-адреса
 - Маршрутизация с использованием IP
- Приватные сети и настройка IP
 - Приватные диапазоны IP-адресов
 - Маска подсети
 - DHCP – динамическое назначение IP
- Доменные имена и система DNS
 - Доменные имена
 - DNS (Domain Name System)
- Транспортный уровень: доставка данных
 - Основные транспортные протоколы
 - Пример сравнения
- Дополнительные детали маршрутизации и работы сети
 - Маршрутизация в Интернете
 - Приватные сети и взаимодействие с глобальной сетью
- Итоговая структура
- Сетевые протоколы и клиент-серверное взаимодействие
 - Прикладные сетевые протоколы
 - Основные примеры
 - Особенности
 - Пример запроса к сайту
 - Уровни взаимодействия (OSI model)
 - Реализация в Unix-системах
 - Сокеты (Sockets)
 - Особенности
 - Ключевые термины
- Операционные системы и процесс загрузки
 - Что такое ОС?
 - 1. Ресурсный менеджер
 - 2. Виртуальная машина
 - Состав ОС
 - Процесс загрузки
 - Этапы включения компьютера

- Файловая система ext2
 - 1. Разделы (Partitions) с точки зрения ОС
 - 2. Виртуальные файловые системы (VFS)
 - 3. Inode (Индексный дескриптор)
 - 4. Структура ext2: Блоки и группы блоков
 - Компоненты группы блоков:
 - 5. Связь файловых систем
 - 6. Ссылки на блоки данных в inode
 - Пример расчёта максимального размера файла (для блока 4 КБ):
- Страничная организация памяти
 - Определение
 - Историческая справка
 - Flat Mode
 - Современная организация памяти
 - Трансляция адресов
 - Пример:
 - Переполнение Frame-ов
 - Проблемы
 - Объём памяти
 - Решение проблемы
 - Многоуровневые таблицы страниц
 - Кэширование
 - Подробности кэширования

03.02.25

ОСНОВНЫЕ ПОНЯТИЯ СЕТЕЙ

Предположим, что у нас есть несколько каналов.

На устройстве, имеющем сетевой адаптер, существует несколько каналов связи.

GSM - СОТОВАЯ СЕТЬ

GSM представляет собой сотовую сеть (например, 3G, 4G и т.д.).

Пример: если мы хотим позвонить из города **A** в город **B**, сигнал передается через несколько узлов сети:

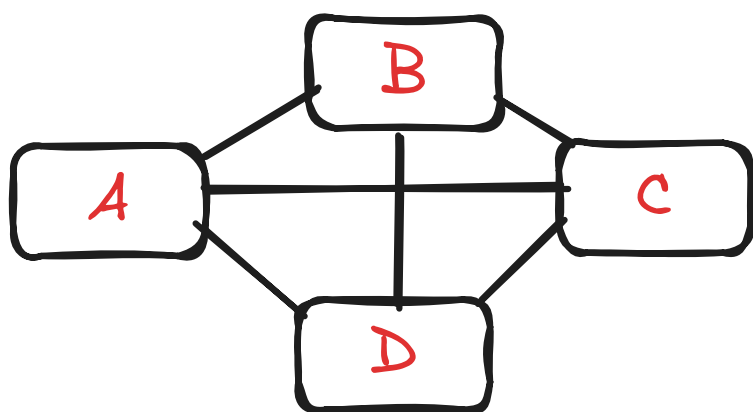
A →  →  →  → B

ДРУГИЕ ВИДЫ СЕТЕЙ

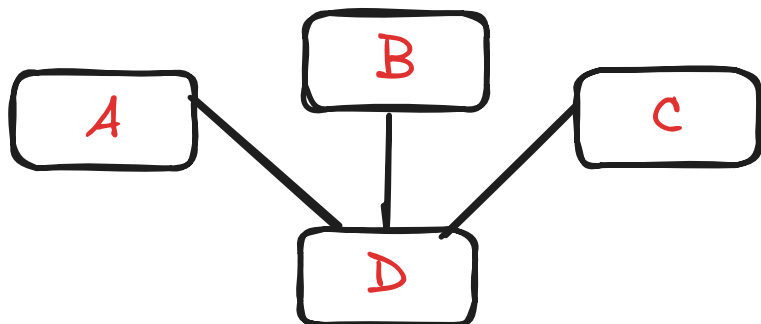
- *Bluetooth* - это тоже сеть.
- *Интернет* - глобальная сеть.
- *NFC* (оплата картой) - тоже сеть.
- *Сеть* - это совокупность устройств, соединённых между собой.
- *Топология* - это способ, которым именно соединены устройства сети.

Виды топологий сети

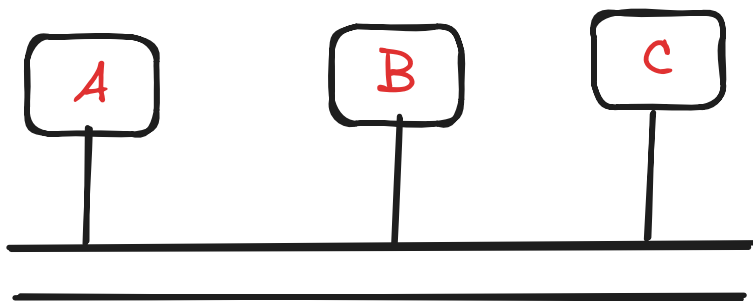
- Все устройства подключены к одной общей линии связи. Если одна связь прорвется, то можно найти другой путь.



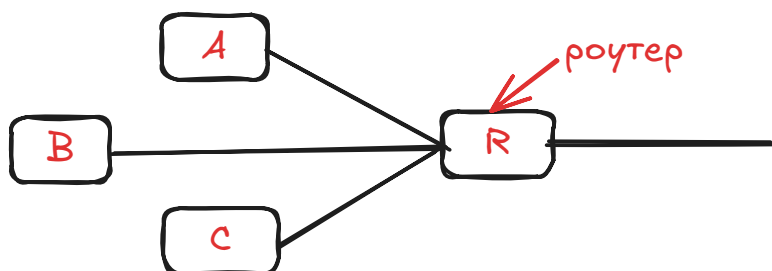
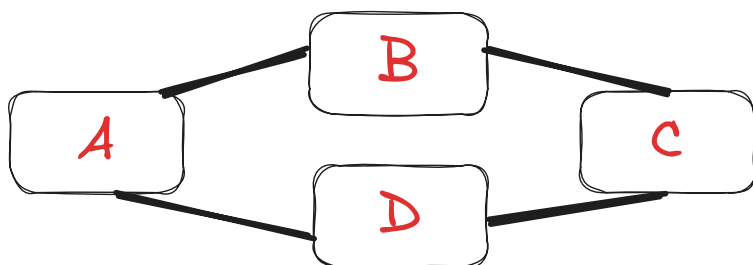
- Из A в B можно через D.



- Можно присоединить к шине.



- Можно и по кругу.



- Передача данных через кабель или радио волну является одним и тем же процессом с технической точки зрения.
Люди придумали разбиение информации на уровни:
- Если станция A и B обмениваются данными, но B удалена, то передача идёт через промежуточные узлы.
- *Высокий уровень напряжения* (например, 5V или 3.3V) интерпретируется как логическая "1".
- *Низкий уровень напряжения* (например, 0V) интерпретируется как логический "0".

ОСНОВНЫЕ ПОНЯТИЯ

- *Модуляция* — процесс изменения параметров сигнала для передачи информации.
- *Демодуляция* — процесс обратного преобразования сигнала.

- **Модуляция-демодуляция (Модем)** — устройство, преобразующее цифровые сигналы в аналоговые и наоборот.

Протокол – это набор правил и стандартов, которые определяют способ передачи, приема и обработки данных между устройствами в сети.

Основные характеристики протокола:

- Определяет **формат** данных (как они представляются).
- Устанавливает **правила взаимодействия** между узлами.
- Определяет, **как данные отправляются и принимаются**.

ПРИМЕРЫ СЕТЕВЫХ ПРОТОКОЛОВ:

1. Физический уровень (кабели, сигналы)

- **Ethernet** – передача данных через проводные сети.
- **Wi-Fi** – беспроводное соединение.
- **MAC-адрес (Media Access Control)** — уникальный физический адрес сетевого интерфейса устройства.
 - Он присваивается производителем и хранится в сетевой карте (NIC) устройства.
 - MAC-адрес состоит из **48 бит** (6 байт) и записывается в формате **XX:XX:XX:YY:YY:YY**.
 - Первые 3 байта (OUI) определяют производителя устройства, а последние 3 байта – уникальный идентификатор.
 - MAC-адрес используется **на канальном уровне** (Ethernet, Wi-Fi) для локальной передачи данных.
 - В отличие от IP-адресов, которые могут меняться, MAC-адрес остается постоянным (но может быть изменен программно).

MAC-адреса и их применение:

- **Коммутация (switching)** – коммутаторы используют MAC-адреса для передачи кадров данных в локальной сети.
- **Фильтрация доступа** – MAC-адреса применяются в сетевой безопасности (например, для ограничения доступа в Wi-Fi).
- **Идентификация узлов** – MAC-адрес помогает определить конкретное устройство в сети, даже если его IP-адрес изменился.
- **PPP (Point-to-Point Protocol)** – протокол прямого соединения.

2. Сетевой уровень (маршрутизация)

- **IP (Internet Protocol)** – основа интернета, передает пакеты между узлами.
- **ICMP (ping)** – диагностика соединений.

3. Транспортный уровень (доставка данных)

- **TCP (Transmission Control Protocol)** – надежная передача данных.
- **UDP (User Datagram Protocol)** – передача без подтверждения.

4. Прикладной уровень (конечные сервисы)

- **HTTP/HTTPS** – передача веб-страниц.
- **FTP** – передача файлов.
- **DNS** – преобразование доменных имен в IP.

Самые большие сети - телеком и радио.

СЕТЕВОЙ УРОВЕНЬ

Сетевой уровень отвечает за соединение узлов, которые физически не связаны между собой.

IP-АДРЕСАЦИЯ

- Узлы в сети идентифицируются с помощью **IP-адресов**.
- В зависимости от версии протокола **IP-адрес** имеет длину **4 байта (IPv4)** или **16 байт (IPv6)**.
- В **IPv4** каждый адрес состоит из четырех десятичных чисел (от 0 до 255), разделенных точками (например, **192.168.1.1**).

- В *IPv6* используются шестнадцатеричные числа (например, `2001:0db8:85a3::8a2e:0370:7334`).

Классификация IP-адресов

- *Частные IP-адреса* (например, `192.168.x.x` , `10.x.x.x` , `172.16.x.x` - `172.31.x.x`) используются внутри локальных сетей и не выходят в интернет.
- *Публичные IP-адреса* назначаются провайдером и доступны в глобальной сети.
- `127.0.0.1 (localhost)` зарезервирован для тестирования внутри устройства.

Маршрутизация на сетевом уровне

Маршрутизация – это процесс выбора пути для передачи данных от источника к получателю через сеть.

- IP-адрес выдается *провайдером* или локальным маршрутизатором.
- Устройства в одной сети могут обмениваться данными без маршрутизатора.
- Если устройства находятся в разных сетях, используется *маршрутизатор*, который определяет путь передачи данных.

Таким образом, сетевой уровень обеспечивает идентификацию узлов, маршрутизацию данных и эффективное взаимодействие между различными устройствами в сети.

19.02.25

Физические пути соединения

На самом базовом уровне данные передаются по двум основным «трактам»:

- **Кабельное соединение:**
 - Используется провод (например, кабель Ethernet).
 - Данные передаются в виде электрических сигналов.
 - Пример: Если сигнал отсутствует — это может означать 0, а наличие сигнала — 1.
- **Беспроводное соединение (радиоволны):**
 - Информация кодируется в радиоволны.
 - Для передачи данных используется процесс **модуляции**, то есть преобразование цифровых данных в аналоговую форму (радиоволны) и обратно.

На этом уровне также используются **сетевые интерфейсы** (например, Ethernet-чип, Wi-Fi-чип, Bluetooth), которые выбирают и реализуют нужный протокол связи.

Стек протоколов

Стек протоколов можно представить как набор слоёв, где каждый слой выполняет свою задачу и «общается» с соседними слоями через стандартизированные протоколы. Можно сравнить это с цепочкой, где каждый участник передаёт информацию дальше, обеспечивая совместимость и надёжность.

- **Программные драйверы и аппаратный интерфейс:**

Драйверы – это специальные программы, которые позволяют операционной системе работать с аппаратными устройствами. Они отвечают за кодирование (на стороне передатчика) и декодирование (на стороне приёмника) данных, что особенно важно как для кабельных соединений, так и для передачи через радиоволны.

Физический уровень: более подробно

Передача сигналов

- **Кодирование данных:**

На физическом уровне информация представляется в виде сигналов.

- **В проводных системах:**

Пример: «нет тока» = 0, «есть ток» = 1.

Таким образом, устройство А кодирует данные в форму электрических сигналов, а устройство В – декодирует их обратно.

- **В беспроводных системах:**

Данные кодируются в радиоволны (с помощью модуляции) и декодируются на приёмной стороне.

Роль ДРАЙВЕРОВ

- **Аппаратный интерфейс:**

Драйверы управляют работой сетевого интерфейса. Это позволяет правильно интерпретировать сигналы – будь то электрические импульсы или радиоволны – и преобразовывать их в данные, понятные на более высоких уровнях стека протоколов.

Идентификация устройств

- **MAC-адреса:**

- Каждый сетевой адаптер (физический компонент, отвечающий за подключение к сети) имеет уникальный MAC-адрес.
- Этот адрес задаётся производителем и используется для идентификации устройства в рамках локальной сети.

Сетевой уровень и IP-протокол

Сетевой протокол

Для обмена данными между устройствами, которые не всегда находятся в непосредственной близости друг от друга, используется сетевой протокол, чаще всего связанный со стеком TCP/IP.

Маршрутизация данных

- **Прямое соединение:**

Если устройства находятся в одной физической сети, данные могут передаваться напрямую.

- **Маршрутизация через промежуточные узлы:**

Если между устройствами нет прямой связи, данные проходят через один или несколько маршрутизаторов:

- Каждый маршрутизатор анализирует адрес получателя и перенаправляет данные дальше.
- В каждом пакете данных может добавляться информация о пройденном маршруте, что позволяет отслеживать путь (например, с помощью команды `tracert`).

IP-адресация

IPv4 и IPv6

- **IPv4:**

- Адрес состоит из четырёх байтов (например, `1.1.1.1`).
- Каждый байт принимает значения от 0 до 255, что ограничивает общее число уникальных адресов примерно 4 миллиардами.

- **IPv6:**

- Использует 128 бит для адресации, что обеспечивает намного большее число уникальных адресов (на практике используется для расширения адресного пространства).

Формат записи IP-адреса

- Пример: 192.168.1.45
 - Четыре числа разделены точками.
 - Диапазон возможных значений: от 0.0.0.0 до 255.255.255.255.

МАРШРУТИЗАЦИЯ С ИСПОЛЬЗОВАНИЕМ IP

- **Алгоритм маршрутизации:**
Когда устройство А отправляет данные устройству В, маршрутизатор (или цепочка маршрутизаторов) находит В, даже если они находятся в разных подсетях. Каждый промежуточный маршрутизатор направляет данные дальше, проверяя адрес получателя.
- **Локальная маршрутизация:**
Если устройство В находится непосредственно в локальной сети, данные передаются напрямую.

ПРИВАТНЫЕ СЕТИ И НАСТРОЙКА IP

ПРИВАТНЫЕ ДИАПАЗОНЫ IP-АДРЕСОВ

Для локальных сетей используются определённые диапазоны адресов, которые не маршрутизируются в глобальном интернете:

- **127.x.x.x:**
Используется для идентификации самого устройства (например, 127.0.0.1 — localhost). Это полезно для разработки и тестирования.
- **192.168.x.x, 10.x.x.x:**
Примеры частных диапазонов для домашних или корпоративных сетей.

МАСКА ПОДСЕТИ

- **Определение сети:**
Маска подсети – это специальное число, которое определяет, какие IP-адреса находятся в одной сети.
 - Пример: Если IP устройства 192.188.1.45 и маска подсети 255.255.0.0, это означает, что все адреса, начинающиеся с 192.188, считаются находящимися в одной сети.
- **Представление маски:**
Маска записывается в виде четырёх чисел, аналогично IP-адресу. Также её можно представить в виде битовой последовательности, где единицы фиксируют часть адреса сети, а нули – часть для устройств.

DHCP – ДИНАМИЧЕСКОЕ НАЗНАЧЕНИЕ IP

- **Задача DHCP:**
При большом количестве устройств вручную задавать уникальные IP-адреса неудобно.
 - DHCP-сервер (обычно встроенный в роутер) автоматически выдаёт IP-адреса свободным устройствам.
 - Это помогает избежать конфликтов, когда два устройства получают один и тот же адрес.

ДОМЕННЫЕ ИМЕНА И СИСТЕМА DNS

ДОМЕННЫЕ ИМЕНА

- **Структура доменного имени:**
Доменные имена состоят из нескольких уровней, разделённых точками:
 - **Top Level Domain (TLD):** Например, .com, .net, .org, .ru и т.д.
 - **Второй уровень:** Например, в домене xyz.com второй уровень — это "xyz".
Владение доменом происходит на условиях ежегодной аренды.

DNS (DOMAIN NAME SYSTEM)

- **Назначение DNS:**

DNS-система сопоставляет доменные имена с IP-адресами.

- При вводе, например, `google.com`, DNS-сервер возвращает соответствующий IP-адрес.
- В разных регионах может возвращаться разный IP (например, в зависимости от географического распределения серверов).

- **Надёжность и выбор сервера:**

Если не доверять провайдерским DNS-серверам, можно настроить альтернативные, такие как `8.8.8.8` (Google) или `1.1.1.1` (Cloudflare).

ТРАНСПОРТНЫЙ УРОВЕНЬ: ДОСТАВКА ДАННЫХ

На транспортном уровне решается вопрос «как данные доставляются от отправителя к получателю» с учётом необходимости гарантии доставки или скорости.

ОСНОВНЫЕ ТРАНСПОРТНЫЕ ПРОТОКОЛЫ

- **TCP (Transmission Control Protocol):**

- Устанавливает виртуальное соединение между отправителем и получателем.
- Гарантирует, что данные дойдут в целости либо будет сообщено об ошибке.
- Подходит для приложений, где критична целостность данных (например, банковские транзакции).

- **UDP (User Datagram Protocol):**

- Передаёт данные без установления соединения и без подтверждения получения.
- Работает быстрее за счёт меньших накладных расходов.
- Используется в приложениях, где небольшие потери данных не критичны (например, онлайн-игры, видеоконференции).

ПРИМЕР СРАВНЕНИЯ

Представьте, что вам нужно отправить посылку:

- **TCP:** Вы выбираете надёжного курьера с гарантией доставки, где вы получите уведомление, если посылка не дошла.
- **UDP:** Вы отправляете посылку через экспресс-сервис без гарантии, что в случае потери посылки вы узнаете об этом сразу, но доставка происходит быстрее.

ДОПОЛНИТЕЛЬНЫЕ ДЕТАЛИ МАРШРУТИЗАЦИИ И РАБОТЫ СЕТИ

МАРШРУТИЗАЦИЯ В ИНТЕРНЕТЕ

- **Маршрутизаторы:**

Каждое устройство, участвующее в маршрутизации (например, $Rk1$, $R2$, ..., Rn), анализирует IP-адрес получателя и перенаправляет данные дальше.

В пакете данных может храниться информация о каждом промежуточном узле, что позволяет отслеживать путь (команда `traceroute`).

- **Сокращение количества узлов:**

Сети строятся так, чтобы количество промежуточных маршрутизаторов было минимальным. Это снижает задержки при передаче данных.

ПРИВАТНЫЕ СЕТИ И ВЗАИМОДЕЙСТВИЕ С ГЛОБАЛЬНОЙ СЕТЬЮ

- **Изоляция частных сетей:**

Устройства в локальной сети (например, с адресами `192.168.x.x`) могут использовать одинаковые IP-адреса в разных сетях, так как они не видны в глобальном интернете.

- **Маршрутизатор и NAT:**

Для связи с глобальной сетью роутер использует механизм NAT (Network Address Translation), который позволяет устройствам частной сети обмениваться данными с внешним миром.

ИТОГОВАЯ СТРУКТУРА

1. **Физический уровень:**

- Передача сигналов через кабели или радиоволны.
- Кодирование и декодирование данных (электрические сигналы, модуляция радиоволн).
- Использование MAC-адресов для идентификации устройств.

2. **Сетевой уровень и IP-протокол:**

- Формат записи IP-адресов (IPv4 и IPv6).
- Маршрутизация данных от узла к узлу через промежуточные маршрутизаторы.
- Применение масок подсети для разделения сетей.

3. **Настройка сети:**

- Частные диапазоны IP-адресов.
- Автоматическое назначение IP через DHCP.
- Работа NAT для выхода в глобальную сеть.

4. **Доменные имена и DNS:**

- Иерархия доменных имён.
- Преобразование доменного имени в IP-адрес через систему DNS.

5. **Транспортный уровень:**

- Выбор протоколов TCP и UDP в зависимости от требований (гарантия доставки vs. скорость).

24.02.25

СЕТЕВЫЕ ПРОТОКОЛЫ И КЛИЕНТ-СЕРВЕРНОЕ ВЗАИМОДЕЙСТВИЕ

ПРИКЛАДНЫЕ СЕТЕВЫЕ ПРОТОКОЛЫ

ОСНОВНЫЕ ПРИМЕРЫ

- **HTTP(S)** - HyperText Transfer Protocol (с шифрованием)
- **FTP** - File Transfer Protocol
- **SMTP** - для отправки электронной почты
- **POP3/IMAP/Exchange** - для получения почты
- **SSH** (Secure Shell) - безопасное выполнение команд на удалённом компьютере

ОСОБЕННОСТИ

- Работают поверх транспортных протоколов (TCP)
- Стандартные порты:
 - HTTP: 80
 - HTTPS: 443
 - FTP: 21
 - SSH: 22
- Порты **1-1024** - привилегированные (требуют прав администратора)

ПРИМЕР ЗАПРОСА К САЙТУ

При вводе `gau.am` в браузере:

1. **DNS-запрос** → получение IP-адреса вида `a.b.c.d`
2. **Установка TCP-соединения:**
 - Клиент: `IP1:port1`
 - Сервер: `IP2:port2` (endpoint)
3. **Обмен данными** по HTTP/HTTPS (текстовые запросы/ответы)
4. **Обработка ответа** (например, статус `200 OK` → рендеринг HTML)

УРОВНИ ВЗАИМОДЕЙСТВИЯ (OSI MODEL)

Уровень	Описание
4. Физический	Передача битов
3. IP	Маршрутизация пакетов
2. TCP	Надёжная доставка данных
1. Прикладной	HTTP, FTP, SMTP

Пример маршрутизации:

Сообщение от A → B через промежуточные узлы. Каждый узел:

1. Если адресат совпадает → обрабатывает
2. Если нет → пересылает дальше

РЕАЛИЗАЦИЯ В UNIX-СИСТЕМАХ

СОКЕТЫ (SOCKETS)

```
// Создание сокета
int socket_fd = socket(AF_INET, SOCK_STREAM, 0);

// Привязка к адресу
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(8080); // Порядок байт (Big Endian)
addr.sin_addr.s_addr = INADDR_ANY; // Слушаем все интерфейсы

bind(socket_fd, (struct sockaddr*)&addr, sizeof(addr));

// Прослушивание входящих соединений
listen(socket_fd, backlog);

// Принятие входящего соединения
int client_fd = accept(socket_fd, (struct sockaddr*)&client_addr, &client_addr_len);
```

- **Аргументы функции `socket`** :
 - `domain` (например, `AF_INET` для IPv4)
 - `type` (например, `SOCK_STREAM` для TCP)
 - `protocol` (обычно `0`, так как он определяется автоматически по первым двум аргументам)
- **Аргументы функции `listen`** :
 - `socket_fd` - файловый дескриптор сокета
 - `backlog` - максимальное количество ожидающих соединений
 - Если `backlog = 0`, используется максимально допустимое значение (зависит от системы)
- **Аргументы функции `accept`** :
 - `socket_fd` - файловый дескриптор сокета
 - `client_addr` - структура для хранения адреса клиента
 - `client_addr_len` - размер структуры адреса

ОСОБЕННОСТИ

- **Клиент-серверная архитектура:**
 - Сервер: один основной процесс
 - Клиенты: множественные подключения
- **Типы сокетов:**
 - `SOCK_STREAM` (TCP)
 - `SOCK_DGRAM` (UDP)
- **Динамические порты** для клиентов (1025-65535)

КЛЮЧЕВЫЕ ТЕРМИНЫ

- **Endpoint** - пара `IP:port`
- **Socket** - программный интерфейс для сетевого взаимодействия
- **Big Endian** - порядок байт для сетевых протоколов (преобразование через `htons()/htonl()`)

03.03.25

ОПЕРАЦИОННЫЕ СИСТЕМЫ И ПРОЦЕСС ЗАГРУЗКИ

Что такое ОС?

1. РЕСУРСНЫЙ МЕНЕДЖЕР

Управляет ключевыми компонентами:

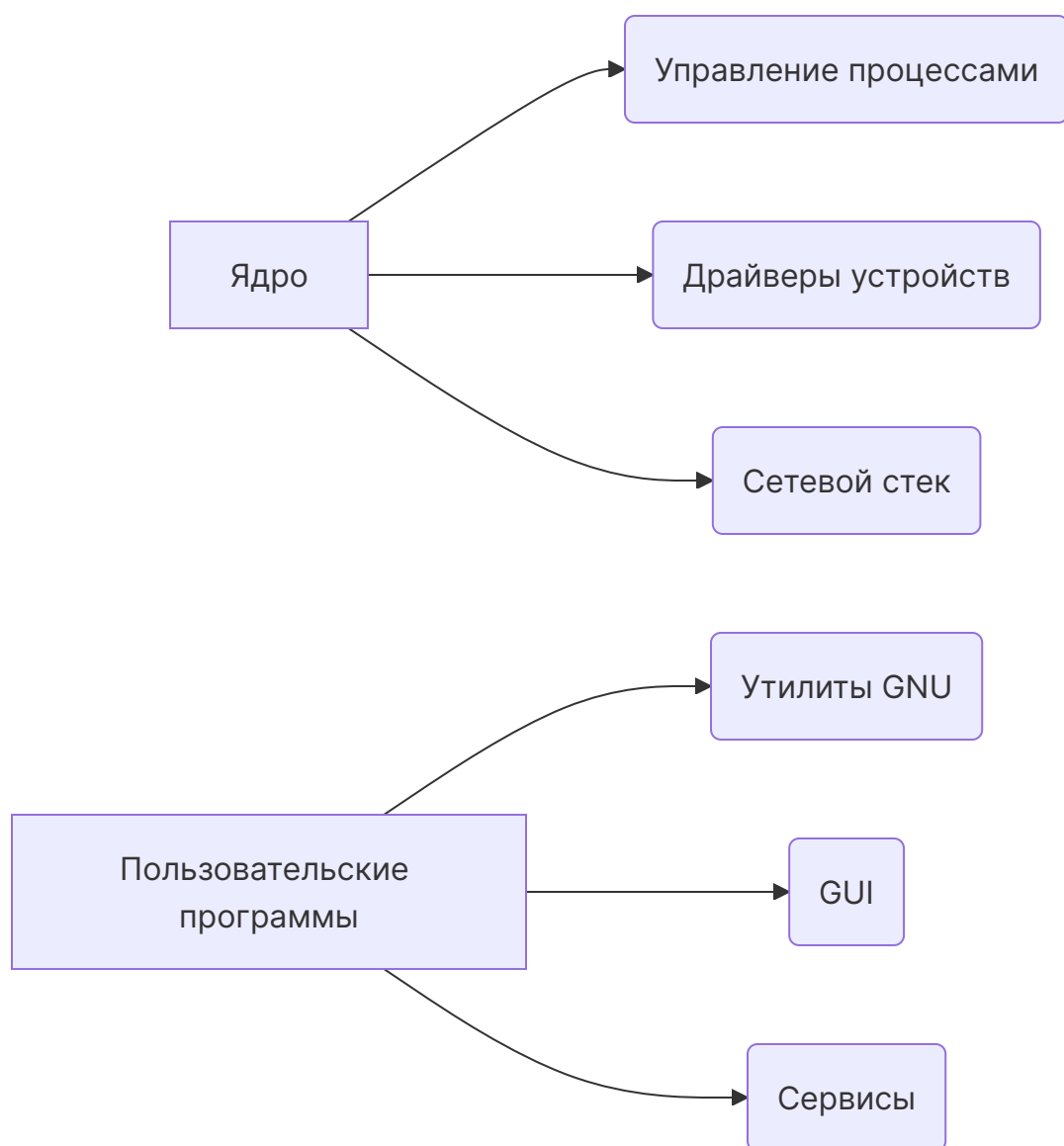
- **CPU** (распределение времени процессов)
- **RAM** (управление памятью)
- **HDD** (файловая система)
- **Устройства** (драйверы для периферии)

2. ВИРТУАЛЬНАЯ МАШИНА

Абстрагирует аппаратные ресурсы:

- **Виртуальная память** → "обманка" между физической RAM и процессами
- **Виртуальные устройства** → унифицированный интерфейс (например, `/dev/input` в Linux)
- **API для программ** → системные вызовы (`read/write`, `fork`, `exec`)

Состав ОС



Примеры:

- Ubuntu = Ядро Linux + Программы GNU
- Windows = Ядро NT + Системные службы

ПРОЦЕСС ЗАГРУЗКИ

ЭТАПЫ ВКЛЮЧЕНИЯ КОМПЬЮТЕРА

1. Активация BIOS/UEFI

BIOS - Basic Input/Output System (устарел)

Unified Extensible Firmware Interface (современная замена BIOS)

- Микропрограмма в ROM (read-only memory) материнской платы
- Сканирует устройства через **SMART-запросы**
- Адресное пространство: `0xFFFFFFF0` (первая инструкция)

2. Поиск загрузочного устройства

- Проверка первых 512 байт диска (в зависимости от последних двух байтов определяется bootable или не bootable устройство)

3. Загрузка boot loader

- Примеры: GRUB (Linux), Windows Boot Manager
- Этапы:
 - Загрузка в RAM в зависимости от пункта 2.
 - Парсинг таблицы разделов
 - Выбор ОС (меню dual-boot)

4. Инициализация ядра ОС

- Загрузка ядра в память (`vmlinux` в Linux)
- Инициализация драйверов и подсистем

5. Запуск init-процесса

- PID 1 в Unix-системах
- Примеры:
 - `systemd` (современные Linux)
 - `launchd` (macOS)
 - `smss.exe` (Windows)
- Запуск остальных процессов

ФАЙЛОВАЯ СИСТЕМА EXT2

1. РАЗДЕЛЫ (PARTITIONS) С ТОЧКИ ЗРЕНИЯ ОС

- **Определение:** Раздел (partition) — логическая область на диске, рассматриваемая ОС как отдельное устройство.
- **Пример:** `/dev/sda1` — первый раздел на диске `sda`.
- **Особенность:** Каждый раздел форматируется под конкретную файловую систему и имеет собственную структуру данных.

2. ВИРТУАЛЬНЫЕ ФАЙЛОВЫЕ СИСТЕМЫ (VFS)

- **Определение:** Абстракция, предоставляющая единый интерфейс для взаимодействия с различными файловыми системами.
- **Примеры физических ФС:**
 - ext2, ext3, ext4 (Linux);
 - NTFS, FAT32 (Windows);
 - APFS (macOS).
- **Выбор ext2:** Базовая структура ext2 сохраняется в ext3/ext4 (журналирование и дополнительные функции не меняют фундаментальные принципы).

3. INODE (ИНДЕКСНЫЙ ДЕСКРИПТОР)

- **Определение:** Структура метаданных файла, не включающая его имя и содержимое.
- **Состав inode:**
 1. Уникальный идентификатор (id);
 2. Временные метки (создание, изменение, доступ);
 3. Владелец (user/group);
 4. Права доступа (permissions);
 5. Размер файла (size);
 6. Количество жёстких ссылок (hard links);
 7. Тип файла (регулярный, каталог, символьное устройство и т.д.);
 8. Ссылки на блоки данных (15 полей, см. п. 6).
- **Важно:**
 - Имя файла хранится в данных каталога, а не в inode.
 - Жёсткие ссылки (hard links) — записи в каталогах, указывающие на один inode. Они не могут пересекать границы файловой системы, так как inode уникальны в пределах одной ФС.

4. СТРУКТУРА EXT2: БЛОКИ И ГРУППЫ БЛОКОВ

- **Блок:** Минимальная единица хранения данных (например, 4 КБ).
- **Группа блоков:** Логическое объединение блоков для оптимизации размещения данных и метаданных.

КОМПОНЕНТЫ ГРУППЫ БЛОКОВ:

1. **Суперблок (Superblock):**
 - Содержит глобальную информацию о ФС (размер, кол-во блоков, свободное пространство и т.д.).
 - Расположен в первых 512 байтах первого блока.
 - Резервные копии суперблока хранятся в других группах блоков.
2. **Таблица дескрипторов групп блоков (BGDT):**
 - Описывает расположение метаданных в каждой группе блоков (например, адреса битовых карт и таблицы inode).
3. **Битовые карты (Bitmaps):**
 - **Inode Bitmap:** Отмечает занятые/свободные inode (1 бит = 1 inode).

- **Data Bitmap**: Аналогично для блоков данных.

4. **Таблица inode (Inode Table)**: Массив inode, каждый из которых соответствует файлу.

5. **Блоки данных (Data Blocks)**: Хранят содержимое файлов и структуры каталогов.

5. СВЯЗЬ ФАЙЛОВЫХ СИСТЕМ

- В одной ФС могут существовать **подмонтированные** файловые системы (например, `/mnt/usb`), но они остаются изолированными.
- **Жёсткие ссылки** не могут указывать на файлы в других ФС, так как inode уникальны только в пределах своей ФС.

6. ССЫЛКИ НА БЛОКИ ДАННЫХ В INODE

Поле `links group` в inode содержит 15 элементов для адресации блоков:

1. **12 прямых ссылок** (direct links) — указывают непосредственно на блоки данных.
2. **Одноуровневая косвенная ссылка** (indirect link) — указывает на блок, содержащий массив адресов блоков данных.
3. **Двухуровневая косвенная ссылка** (double indirect link) — адресует блок, который ссылается на другие блоки с адресами.
4. **Трёхуровневая косвенная ссылка** (triple indirect link) — для очень больших файлов.

ПРИМЕР РАСЧЁТА МАКСИМАЛЬНОГО РАЗМЕРА ФАЙЛА (для блока 4 КБ):

- **Прямые ссылки**: $12 \times 4 \text{ КБ} = 48 \text{ КБ}$.
- **Одноуровневая**: $(4 \text{ КБ} / 4 \text{ байта}) \times 4 \text{ КБ} = 1024 \times 4 \text{ КБ} = 4 \text{ МБ}$.
- **Двухуровневая**: $1024 \times 1024 \times 4 \text{ КБ} = 4 \text{ ГБ}$.
- **Трёхуровневая**: $1024^3 \times 4 \text{ КБ} = 4 \text{ ТБ}$.
- **Итого**: $48 \text{ КБ} + 4 \text{ МБ} + 4 \text{ ГБ} + 4 \text{ ТБ}$.

24.03.25 + 31.03.25

СТРАНИЧНАЯ ОРГАНИЗАЦИЯ ПАМЯТИ

ОПРЕДЕЛЕНИЕ

Страница памяти — это механизм разделения виртуальной памяти для отображения в оперативную память.

ИСТОРИЧЕСКАЯ СПРАВКА

Ранее в архитектурах x86-64 использовалось сегментное хранение. Данные, где находились адреса сегментов, назывались **segment table**. Однако этот подход оказался неэффективным при большом количестве процессов, так как:

- Необходимо было держать все куски сегментов в RAM, либо ничего не держать.

FLAT MODE

Flat Mode — это режим, при котором все сегменты начинаются и заканчиваются в одном и том же месте. Таким образом, адрес `segment offset` становится физическим адресом. Это эквивалентно одному большому сегменту.

СОВРЕМЕННАЯ ОРГАНИЗАЦИЯ ПАМЯТИ

Сейчас память организуется страничным методом. Каждая страница — это регион памяти, например, размером 4 КБ. В этом случае общее количество страниц можно рассчитать как:

$$\text{Количество страниц} = \frac{2^{32}}{2^{12}} = 2^{20}$$

ТРАНСЛЯЦИЯ АДРЕСОВ

- **Page Frame:** *i*-я страница отображается на *j*-ю в физическом пространстве.
- **Page Table:** В каждом процессе есть таблица страниц (Page Table), где хранятся:
 - Индекс страницы (Page Index)
 - Индекс кадра (Frame Index)
 - Флаги (Flags)

Для нахождения логического адреса:

1. Берётся Page Index.
2. Прибавляется Offset (смещение).

ПРИМЕР:

- **Page Index:** 20 бит.
- **Offset:** 12 бит.

Для адреса в физическом пространстве обращаемся к Page Table и берём Frame Index.

ПЕРЕПОЛНЕНИЕ FRAME-ОВ

При переполнении кадров возможны следующие исходы:

1. **Out Of Memory:** Недостаток памяти.
2. **Swap:** Подключается внешнее устройство (например, диск) с разделом **swap**. Если кадры заняты, выбирается "жертва" (Victim Frame), которая сохраняется в swap. Когда кадр снова нужен, он возвращается в RAM (необязательно на тот же адрес). Местоположение кадра определяется с помощью флагов.

ПРОБЛЕМЫ

- **Фрагментация:** Возникает из-за перемещения данных между RAM и swap.
- **Page Table:** Таблицу страниц нельзя переместить из RAM, так как это приведёт к потере доступа ко всем адресам.

ОБЪЁМ ПАМЯТИ

Для 64-битных систем:

$$\text{Количество страниц} = \frac{2^{64}}{2^{12}} = 2^{52}$$

Это огромный объём памяти, который всегда должен быть доступен в RAM.

РЕШЕНИЕ ПРОБЛЕМЫ

МНОГОУРОВНЕВЫЕ ТАБЛИЦЫ СТРАНИЦ

Вместо одной Page Table используется несколько таблиц. Для хранения используется **Page Directory**. Адресация выглядит следующим образом:

1. Из PTI (Page Table Index) находим номер таблицы.
2. Из PI (Page Index) находим FI (Frame Index).

3. Прибавляем Offset.

Таким образом, в памяти держится только Page Directory.

КЭШИРОВАНИЕ

Для уменьшения нагрузки используется кэш на уровне CPU. Кэш хранит часто используемые адреса, что значительно ускоряет процесс обращения к памяти.

ПОДРОБНОСТИ КЭШИРОВАНИЯ

- **TLB (Translation Lookaside Buffer):** Это специальный кэш, который хранит соответствия между виртуальными и физическими адресами. Он позволяет избежать обращения к Page Table при каждом запросе.
- **Уровни кэша:** Современные процессоры имеют несколько уровней кэша (L1, L2, L3), которые используются для хранения данных и инструкций, включая информацию о страницах памяти.
- **Эффективность:** Использование кэша значительно снижает задержки при обращении к памяти, так как большинство запросов обрабатываются на уровне кэша, а не RAM.